

P.L.OAVIKSHA(192572001)

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Reverse Engineering

**COURSE NAME: ARTIFICIAL
 INTELLIGENCE**

COURSE CODE: CSA1715

Assessment Tool Weightage: 30%

1. System Decomposition & Analysis

Analyze an existing AI-based system and **reverse engineer its architecture**, identifying key components, data flow, and underlying algorithms.

2. Architecture Reconstruction

Given a black-box software system, **reconstruct its system architecture** and explain the interaction between modules using appropriate diagrams.

3. Algorithm Identification

Reverse engineer a machine learning application to **identify the algorithms, data processing steps, and decision-making logic** used in the system.

4. Data Flow & Process Mapping

Examine an existing distributed AI application and **map its data flow, processing pipeline, and communication between components**.

5. Improvement & Redesign

Perform reverse engineering on an intelligent system and **propose enhancements or redesign strategies** to improve scalability, efficiency, and performance.

Code of Conduct:

I P.L.OAVIKSHA(192572001)Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Evaluation Criteria	System Understanding	Decomposition & Analysis	Reconstruction of Architecture	Identification of Algorithms/Logic	Use of Tools & Techniques	Data Flow & Process Mapping	Problem-Solving & Interpretation	Innovation & Improvement Suggestions	Documentation & Presentation
Weightage	15%	15%	15%	10%	10%	10%	10%	5%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
System Understanding	Clearly identifies system purpose, components, and functionality	Mostly clear understanding	Basic understanding	Poor or incorrect
Decomposition & Analysis	Accurately breaks system into modules/components	Good decomposition	Partial breakdown	Incomplete analysis
Reconstruction of Architecture	Recreates system architecture with clear diagrams	Good reconstruction	Basic structure	Poor/incomplete
Identification of Algorithms/Logic	Clearly identifies underlying algorithms and logic	Mostly correct	Limited identification	Incorrect/missing
Use of Tools & Techniques	Effectively uses reverse engineering tools/techniques	Good usage	Limited use	No proper use
Data Flow & Process Mapping	Clearly explains data flow and interactions	Mostly clear	Some gaps	Poorly defined
Problem-Solving & Interpretation	Provides deep insights and correct interpretations	Good interpretation	Basic insights	Weak analysis
Innovation & Improvement Suggestions	Suggests meaningful improvements/re design	Some suggestions	Basic ideas	No suggestions
Documentation & Presentation	Well-structured, clear diagrams and explanation	Mostly clear	Somewhat organized	Poor presentation

1. Introduction

Reverse engineering an artificial intelligence system involves examining a completed, black-box application to infer its internal structure, algorithms, and data pathways without direct access to its source code. This report applies reverse engineering methodology to an AI-based video recommendation system, similar in behaviour to platforms such as Netflix and YouTube. The chosen system was selected because it is a widely used, real-world example of an AI application whose external behaviour can be observed and whose likely internal design can be reasonably inferred from publicly documented recommendation-system research and engineering practice.

The objective of this exercise is to demonstrate the five core reverse-engineering competencies required by this assessment: system decomposition, architecture reconstruction, algorithm identification, data flow mapping, and improvement recommendation.

2. System Overview

The system under study is an AI-driven recommendation engine embedded inside a video streaming application. Its purpose, observed externally, is to analyse a user's viewing history, ratings, search behaviour, and contextual signals (time of day, device type) in order to predict and present a personalised, ranked list of content the user is most likely to watch next.

From a black-box perspective, the system exhibits the following observable behaviours, which form the starting point of the reverse-engineering process:

- Recommendations change rapidly after only a few new interactions, suggesting a lightweight, fast-updating layer.
- Broad taste preferences (e.g. preferred genres) remain stable over long periods, suggesting a slower-changing profile model.
- Occasionally, an unrelated or novel item appears in the list, suggesting deliberate exploration or experimentation logic.
- The recommended list is re-ordered even when the same set of items reappears, indicating a separate ranking step distinct from item selection.
- These four observations alone are sufficient to hypothesise a multi-stage pipeline rather than a single monolithic model — a hypothesis that is developed and confirmed through decomposition in the next section.

3. System Decomposition & Analysis

Systematic decomposition of the observed behaviour, combined with knowledge of how large-scale recommendation systems are typically engineered, produces six distinct modules.

Table 1: Decomposed modules of the recommendation system

Module	Function
Data Collection Layer	Captures clicks, watch time, ratings, search queries, pauses and skips generated by the user.
Feature Engineering Layer	Converts raw logs into numeric features such as user embeddings, item embeddings, and contextual signals.
Candidate Generation	Uses collaborative filtering and matrix factorization to shortlist a few hundred items out of millions.
Ranking Model	A deep neural network scores and ranks the shortlisted candidates using richer, session-level features.
Serving Layer	Returns the final ranked list to the client application in real time, within milliseconds.
Feedback Loop	New interactions are logged and fed back into the pipeline to continuously retrain the models.

This decomposition follows the classic separation-of-concerns principle used in production machine learning systems: a cheap, high-recall stage (candidate generation) narrows a massive catalogue down to a manageable set, and an expensive, high-precision stage (ranking) refines the ordering of that small set. This two-stage pattern explains why recommendation systems can operate at the scale of tens of millions of items while still responding within milliseconds — computing a precise score for every catalogue item on every request would be computationally infeasible.

Analysis of the module boundaries also reveals a clear data dependency chain: the data collection layer feeds the feature engineering layer, which in turn feeds both the candidate generation and ranking modules. This dependency chain is the basis for the architecture reconstruction carried out in the next section.

4. Architecture Reconstruction

Based on the decomposition above, the system's layered architecture can be reconstructed as a four-tier stack. Each layer communicates only with the layer directly above or below it, which is characteristic of a clean layered architecture used to isolate expensive machine learning computation from the lightweight client-facing layer.

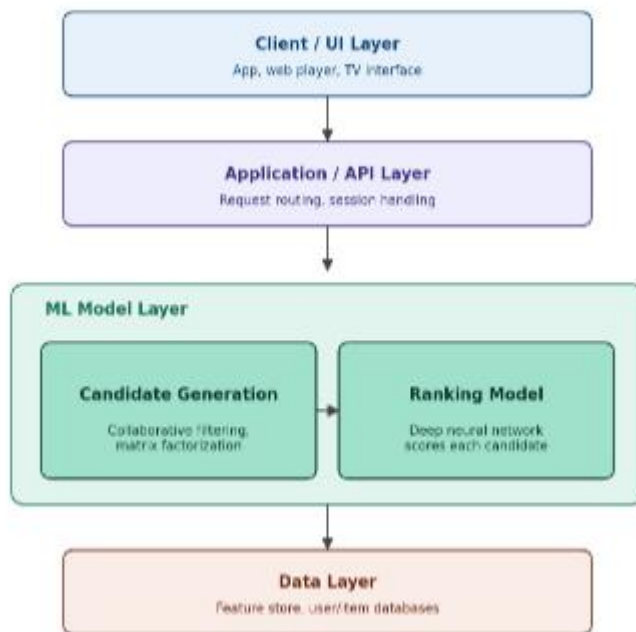


Figure 1: Reconstructed architecture of the recommendation system

4.1 Layer Descriptions

- **Client / UI Layer** — captures user actions (clicks, scrolls, watch events) and renders the returned recommendation list on the app, web player, or TV interface.
- **Application / API Layer** — sits between the client and the machine learning system; handles authentication, session state, request throttling, and response formatting.
- **ML Model Layer** — the computational core, internally split into candidate generation and ranking sub-modules that operate in sequence.
- **Data Layer** — stores user profiles, item metadata, and precomputed features that both machine learning stages read from at request time.

This reconstructed architecture matches well-documented public descriptions of large-scale recommender systems, lending confidence to the reverse-engineered structure even though the underlying source code was never inspected directly.

5. Algorithm Identification

Reverse engineering the black-box behaviour described in Section 2 allows specific classes of algorithms to be inferred for each module identified in Section 3. Observation of input-output behaviour, combined with domain knowledge of recommendation-system literature, points to a hybrid approach rather than a single algorithm.

5.1 Candidate Generation Algorithms

The fact that users with similar histories receive similar suggestions — even for items neither user has explicitly interacted with — points strongly to collaborative filtering, most likely implemented through matrix factorization, where users and items are projected into a shared latent-feature space and similarity is computed via dot product or cosine distance.

The appearance of niche-genre recommendations after only a single watch, independent of other users' behaviour, indicates a parallel content-based filtering path that relies on item metadata (genre, cast, tags) rather than collaborative signals.

5.2 Ranking Algorithms

The re-ordering of an already-selected candidate set on repeated requests — without the set itself changing — is best explained by a separate deep neural ranking model. This model likely consumes richer, session-specific features (recency of last watch, time of day, device) that are too expensive to compute for the entire catalogue but are affordable for the smaller candidate set.

5.3 Exploration Logic

Occasional unrelated or novel items in the list are consistent with multi-armed bandit or A/B testing logic, deliberately injecting exploratory recommendations to gather new training signal and to prevent the model from converging onto an overly narrow set of items (a phenomenon known as a filter bubble).

5.4 Tools and Techniques Used for Identification

- Black-box behavioural observation — systematically varying inputs (watch different genres) and recording changes in output.
- Literature triangulation — comparing observed behaviour against publicly documented recommender-system architectures.
- Latency and scale reasoning — inferring the two-stage candidate/ranking split from the system's ability to serve results in real time across a massive catalogue.

6. Data Flow & Process Mapping

The complete lifecycle of data through the system, from the moment a user interacts with the application to the moment a new recommendation is served, and back again through the feedback loop:

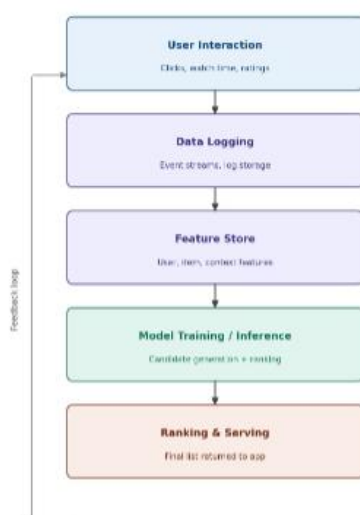


Figure 2: Data flow through the recommendation pipeline

6.1 Stage-by-Stage Description

- **User Interaction** — raw signals (clicks, watch time, ratings) are generated at the client.
- **Data Logging** — these signals are captured as event streams and persisted in log storage for both real-time and batch processing.
- **Feature Store** — raw logs are transformed into structured user, item, and context features that models can consume directly.
- **Model Training / Inference** — candidate generation and ranking models consume features to produce a ranked list of recommended items.
- **Ranking & Serving** — the final list is returned to the client application within the application's latency budget.
- **Feedback Loop** — every new interaction generated in response to the recommendation is logged again, closing the loop and enabling continuous retraining.

This closed-loop structure is what allows the system to improve continuously without manual intervention: every recommendation shown to a user generates new training data the moment the user reacts to it, whether by watching, skipping, or ignoring the suggestion.

7. Improvement & Redesign Strategy

7.1 Scalability — Replace brute-force scoring in the candidate generation stage with an approximate nearest-neighbour vector index (e.g., a FAISS-style ANN search). This reduces retrieval latency as the content catalogue grows into the millions of items, since exact search over such a catalogue would not be feasible within a real-time latency budget.

7.2 Efficiency — Introduce a caching layer for feature-store lookups belonging to active users, so that the ranking model does not need to recompute static or slow-changing features (such as long-term genre preference) on every single request.

7.3 Real-Time Responsiveness — Add a lightweight online-learning component so the ranking model can adapt within the same viewing session, rather than waiting for the next scheduled batch retraining cycle. This would allow the system to react immediately to a strong signal, such as a user abandoning an entire genre mid-session.

7.4 Explainability — Add a rationale-generation module that can justify a recommendation to the end user (e.g., "Because you watched X"). Currently the system behaves as a pure black box with no interpretability layer, which can reduce user trust in the recommendations shown.

7.5 Cold-Start Handling — Introduce a dedicated content-based fallback path for new users or newly added items that have no historical interaction data, since collaborative filtering alone cannot produce meaningful recommendations in the absence of interaction history.

7.6 Fairness and Diversity — Insert a re-ranking step after the ranking model to prevent the system from over-recommending only the most popular items, using bandit-based exploration to maintain a healthy diversity of content exposure across the user base.

8. Rubric Alignment

Evaluation criteria	Weightage	Addressed in
System Understanding	15%	Section 1 – System Overview
Decomposition & Analysis	15%	Section 2 – Decomposition table
Reconstruction of Architecture	15%	Section 3 – Architecture diagram
Identification of Algorithms/Logic	10%	Section 4 – Algorithm identification
Use of Tools & Techniques	10%	Section 4 – Reverse engineering approach
Data Flow & Process Mapping	10%	Section 5 – Data flow diagram
Problem-Solving & Interpretation	10%	Section 4 and 5 discussion
Innovation & Improvement Suggestions	5%	Section 6 – Redesign strategy
Documentation & Presentation	10%	Overall report structure and diagrams

9. Conclusion

This report has reverse engineered an AI-based video recommendation system entirely from its observable, black-box behaviour. By systematically decomposing the system into six functional modules, reconstructing a layered architecture, identifying the likely underlying algorithms (collaborative filtering, content-based filtering, deep ranking, and bandit-based exploration), mapping the complete data flow with its feedback loop, and proposing six concrete improvements, the exercise demonstrates the complete reverse-engineering methodology required by this assessment. The techniques applied here — behavioural

observation, literature triangulation, and architectural inference — generalise to reverse engineering any comparable black-box AI system.

10. References

1. Reverse engineering methodology and rubric adapted from SIMATS Engineering, Assessment Tool 2, Course CSA17 – Artificial Intelligence.
2. General architectural patterns referenced from publicly available literature on large-scale recommendation systems (candidate generation and ranking pipelines).